

Pumping Lemma per i linguaggi regolari

venerdì 18 marzo 2022 15:44

- UN LINGUAGGIO A È REGOLARE, SE LO POSSO CARATTERIZZARE CON UN DFA, NFA, GNFA O UNA REG.

DIMOSTRARE CHE UN CERTO LINGUAGGIO A È REGOLARE.

- si potrebbe mostrare un automa DFA che riconosca quel linguaggio.
- si potrebbe mostrare un NFA o GNFA o una REG.

questo è "EASY".

Ma la vera difficoltà è dimostrare che A NON È UN LINGUAGGIO REGOLARE!

come si fa?

significa che non esiste GNFA, DFA o REG che cattura quel linguaggio.

Dimostrare che non esiste qualcosa non significa prendere un automa e fare vedere che non riconosce il linguaggio e quindi non è regolare, al contrario bisogna dimostrare che, qualunque sia l'automa a stati finiti, non può riconoscere il linguaggio.

Questo è difficile!

ESEMPIO DI UN LINGUAGGIO che potrebbe non essere REGOLARE

$$L = \{ 0^m 1^m \mid m \geq 0 \}$$

$$\Sigma = \{ 0, 1 \}^*$$

PRIMA ci sono m ZERI e poi ci sono m UNI.

se $m=0$, $\epsilon \in L$.

se $m=1$, $01 \in L$

se $m=2$, $0011 \in L$

$101 \notin L$

$1100 \notin L$

$00111 \notin L$

$\rightarrow \underbrace{0000\dots}_{m\text{-Volte}} \underbrace{1111\dots}_{m\text{-Volte}}$

$\rightarrow$ questo è una stringa appartenente al linguaggio.

✓

✗

QUESTO LINGUAGGIO È REGOLARE?

Può esistere un DFA che può riconoscere questo linguaggio?

IL NUMERO degli stati di un DFA è finito!

ho 5 dita di una mano e ho un solo dito selezionato ogni volta, m è grande.

Supponiamo che esista un automa che riconosca questo linguaggio con 1.000.000 di stati, bene questa stringa potrebbe avere 1.000.001 di zeri e 1.000.001 di uni.

NON DOVREBBE ESSERE POSSIBILE PER QUESTO AUTOMA RICONOSCERE IL LINGUAGGIO, perché ho un numero di stati troppo piccolo rispetto alla stringa di ingresso.

$$\Sigma = \{ 0, 1 \}^* \quad 2^? \text{ è tantissimo.}$$

invece se fosse stato:

$$\Sigma = \{ 0, 1 \} \quad 2^2 = 4 \text{ (STATI)}$$

ESEMPIO DI UN LINGUAGGIO che potrebbe non essere REGOLARE

$$L = \{ w \in \{ 0, 1 \}^* \mid w \text{ ha un numero uguale di } 0 \text{ e } 1 \}$$

QUESTO LINGUAGGIO È REGOLARE?

$\epsilon \in L$
 $10 \in L$
 $01 \in L$

$01011 \notin L$

✗

✓

È lo stesso discorso di prima, sembrerebbe di no.

devo contare gli zeri e gli uni, ma superato un certo numero di stati dell'automa, non posso più contare.

Questo linguaggio sembra stringa di lunghezza nell'intervallo finito.

QUESTO LINGUAGGIO CONTIENE STRINGHE DI LUNGHEZZA ARBITRARIAMENTE LUNGA.

SEMBREREBBE ESSERE ANCORA NO LA RISPOSTA.

ESEMPIO DI UN LINGUAGGIO CHE POTREBBE NON ESSERE REGOLARE

$$D = \{w \in \{0,1\}^* \mid w \text{ ha un numero uguale di "01" e "10" come sottostringhe}\}$$

$$\begin{array}{l} \varepsilon \in D \\ 010 \in D \\ \boxed{010} \\ 11 \rightarrow 1 \text{ occorrenza di "01" e di "10".} \end{array}$$

✓

$$\begin{array}{l} 0101 \notin D \\ \boxed{0101} \\ \rightarrow 2 \text{ occorrenze di "01"} \\ 1 \text{ occorrenza di "10"} \end{array}$$

✗

QUESTO LINGUAGGIO È REGOLARE?

anche qui devo contare le sottostringhe di "01" e "10".

Lo stesso problema, la risposta potrebbe essere "NO".

INVECE È SBAGLIATA.

LA RISPOSTA alla domanda È SÌ!

È Regolare!

UN RAGIONAMENTO INTUITIVO, NON BASTA, VA FATTA UNA DIMOSTRAZIONE RIGOROSA.

INTRODUCIAMO UN ELEMENTO NUOVO CHE CI CONSENTE DI EFFETTUARE LA DIMOSTRAZIONE!

STRUMENTO DI BASE PER DIMOSTRARE LA "NON REGOLARITÀ" DI UN LINGUAGGIO!

Pumping Lemma

PUMPING LEMMA

PER I LINGUAGGI REGOLARI.

PUMPING = POMPARE, AUMENTARE O DIMINUIRE UNA CERTA DIMENSIONE.

ENUNCIATO: se A è un linguaggio regolare, allora esiste un certo valore $p > 0$ (questo numero è la lunghezza di pompaggio - The pumping length) dove:

se S è una stringa del linguaggio, $S \in A$, tale che:

- $|S| \geq p$ (la lunghezza di S è $\geq$ di p)

allora, esiste una suddivisione di questa stringa S in tre parti, che chiamiamo X , Y e Z :

$$S = \underline{XYZ}$$

→ sono sottostringhe in cui si può decomporre l'elemento del linguaggio S dove valgono queste tre condizioni:

1. per ogni $i \geq 0$, $XY^iZ \in A$.

QUESTO È IL POMPARE, la parte Y della stringa la posso pompare, replicare, quante volte voglio, e quello che ottengo è ancora un elemento del linguaggio A .

2. $|Y| > 0$, Y non può essere la stringa vuota.

$$\rightarrow 3. |XY| \leq p$$

Questo lemma si applica ai linguaggi regolari, quindi, come faremo ad utilizzarlo per un linguaggio che non è regolare?

PER ASSURDO; Se il linguaggio fosse regolare dovrebbe valere il Pumping lemma.

Quindi, faccio una serie di ragionamenti dimostrando che, in realtà, il pumping lemma non vale, ergo, il linguaggio non è regolare.

PER DIMOSTRARE IL CONTRARIO DEVO DIRE CHE, ESISTE UNA STRINGA S , IN CUI OGNI SUDDIVISIONE NON VALE.

CAPIAMO BENE

• se ho una stringa S la posso vedere come costituita da 3 parti:

se la stringa è più lunga di P la posso vedere costituita da 3 parti:

S : x y z

ogni stringa abbastanza lunga nel linguaggio la posso spezzare in 3 parti e poi, IL TEOREMA dice, che se prendo la stessa stringa e ci tolgo la y , la stringa rimanente fa ancora parte del linguaggio.

S : x z $\in A$ se $i=0$, $y^0 = \epsilon$

Perché corrisponde al primo caso nel caso in cui $i=0$, perché $i \geq 0$!

se concateno y
+ se stessa 0
volte viene la
stringa vuota.

IL PUMPING lemma funziona anche pompando verso il basso, caso $i=0$.

MA se io prendo sempre la stringa S e la pompo verso l'alto, la stringa rimanente fa ancora parte del linguaggio.

S : x y y z

corrisponde al primo caso nel caso $i=2$.

...

funziona anche con $i=10^6$, per esempio, la stringa S fa sempre parte del linguaggio.

NON C'È LIMITE.

ORA, IL PUMPING lemma, Bisogna dimostrarlo:

DIM

se A è regolare esiste un DFA M che riconosce questo linguaggio: $L(M) = A$.

Questo automa M è fatto da un certo numero di stati: Q , e supponiamo che il numero di stati sia P :

$$|Q| = P$$

Prendiamo una stringa S che fa parte del linguaggio, $S \in A$, $|S| \geq P$, la cui lunghezza è $\geq P$.

Se $S \in A$, se eseguo $M(S)$ accetta.

se M accetta vuol dire che (definizione di accettazione di un dfa) esiste una sequenza di stati

$$\begin{array}{c} R_0, R_1, R_2, \dots, R_n \in Q \\ \parallel \quad \downarrow \downarrow \\ q_0 \quad \in F \text{ (stati di accettazione)} \end{array}$$

IL PUNTO È CHE IN UN DFA, IN OGNI PASSAGGIO, CONSUMO ESATTAMENTE UN SIMBOLO DELL'INPUT.

n = lunghezza della stringa

$$q \geq q$$

$$q+1 \geq q$$

$$R_0, R_1, R_2, \dots, R_n \in Q$$

questa sequenza è lunga $n+1$

MA SICCOME $n \geq P$, allora:

$$n+1 \geq P+1$$

→ Principio della piccionaia!

se ho m piccioni ed $m-1$ pane, e tutti si rifugano dentro la tana, non so dove, ma ci sarà almeno 1 tana che conterrà almeno due piccioni.

se qui ho $P+1$ stati almeno, io non so dove sta, ma siccome ho solo P stati, uno stato si deve ripetere.

Perciò alla fine ci saranno degli stati R_j e R_k identici ad un altro stato q .

$R_0, R_1 \dots R_j R_k$
 $P+1$

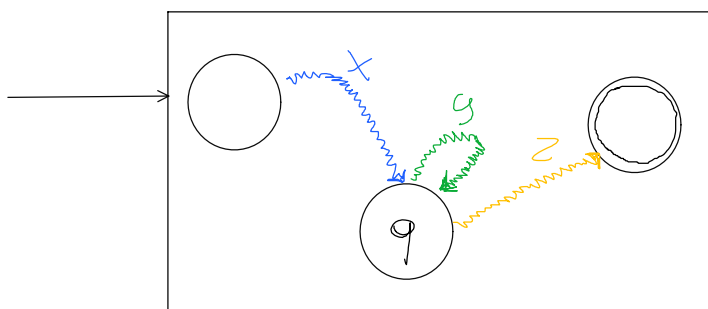
E so che $K \leq P$

COSA SUCCEDDE?

IL NOSTRO AUTOMATO COMINCIA CON UNO STATO INIZIALE R_0 , POI COMINCIA A CONSUMARE, CONSUMANDO FA UNA SERIE DI TRANSIZIONI E ARRIVA AL NOSTRO STATO q , MA QUESTO STATO q SI RIPETE, MA POI QUESTO AUTOMATO DOVRÀ ACCETTARE E QUINDI DEVE PASSARE IN UNO STATO DI ACCETTAZIONE.

quindi se effettivamente $R_0, R_1 \dots R_m$ è la mia sequenza di $|S|$ che mi viene consumata allora posso identificare con:

- $X = S_1 \dots S_j$ fino a q
- $Y = S_{j+1} \dots S_k$ loop dentro al nodo q .
- $Z = S_{k+1} \dots S_m$ fino allo stato di accettazione.



io posso applicare la transizione in un dfa se $\delta(R_i, S_{i+1}) = R_{i+1}$.

ORA CONSIDERIAMO LA STRINGA XZ , OVE TUTTI I SIMBOLI $S_1 \dots S_j S_{k+1} \dots S_m$
 $\downarrow \quad \downarrow$
 $X \quad Z$

ho pompato verso il basso, ho tolto la parte Y .
LA MIA TESI È CHE, LA STESSA MACCHINA, SU X E Y , ACCETTA.

$M(XY)$ Accetta.

L'automa è un DFA, quindi quando legge quella sequenza fa: $R_0 \rightarrow R_1 \dots \rightarrow R_j$

ORA, PER POTER ACCETTARE:

$$\delta(R_k, S_{k+1}) \rightarrow R_{k+1}$$

$$[S_k \rightarrow S_{k+1}]$$

$$\downarrow$$

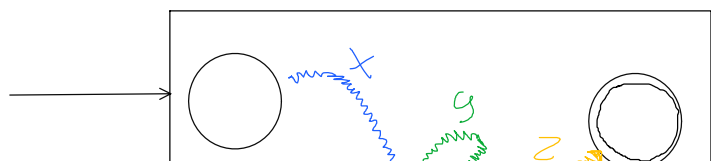
$$R_k = R_j$$

$$\delta(R_j, S_{k+1}) \rightarrow R_{k+1}$$

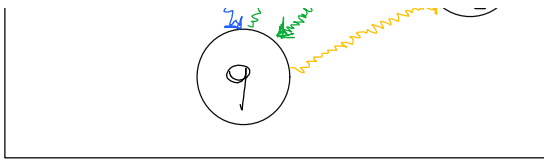
che vuol dire? che quando sono arrivato qui, posso mangiarmi S_{k+1} e andare in Z .

$R_0 \rightarrow R_1 \dots \rightarrow R_j \rightarrow R_{k+1} \rightarrow \dots \rightarrow R_m$ // COSÌ RICONOSCO LA STRINGA XZ , $M(XZ)$.

CONCLUSIONI



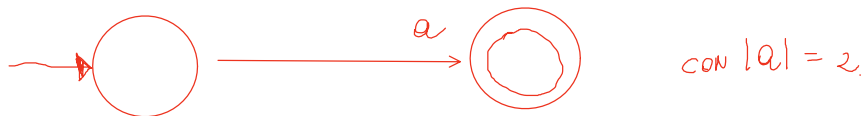
S
 $||$
INPUT: $XYYZ$ viene accettata (faccio il ciclo Y due volte)
INPUT: $X Y^{10^6} Z$ viene accettata (faccio il ciclo 10^6 volte per Y)



ovviamente la stringa deve essere grande.

ESEMPIO

Voglio poter accettare una stringa di lunghezza 2.

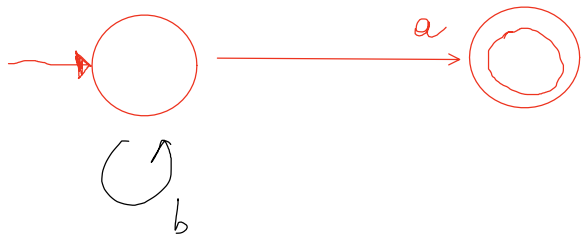


Accettore una $|S| = 2$ di lunghezza 2!

SVLG

Con un automa fatto così non posso accettare una stringa di lunghezza 2, perché entro dentro l'automa, consumo una stringa e accetto.

Se la stringa è corta, il lemma non vale, per poter accettare quella stringa deve esserci un ciclo:



IL TEOREMA dice che, per ogni stringa abbastanza lunga, se la accetti, ci deve essere un ciclo nel grafo, perché altrimenti non puoi accettarla.

E se c'è un ciclo nel grafo, allora devi accettare non solo quella stringa ma anche quella in cui il ciclo non lo segui e quella dove il ciclo lo segui un numero arbitrario di volte.

Se la stringa è abbastanza lunga eccede il numero degli stati di un DFA, quindi dentro questo automa ci deve essere un ciclo, questo ciclo può essere percorso 0, 1, 2, 10⁶ volte, questa parte del ciclo consuma gli input che corrispondono alla parte y della nostra stringa: $|S| = XYZ$

quindi $S \in L$ per qualunque i .

questo è tutto sul pumping lemma!